



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра информационных технологий в атомной энергетике

Практическая работа 3

по дисциплине «Информационные системы управления данными и документами»

Студент группы ИКБО-50-23

Враженко Д. О.

(подпись студента)

Принял преподаватель

Тесла-Блозва С. А.

(подпись руководителя)

Работа представлена

«___» _____ 2026 г.

Допущен к работе

«___» _____ 2026 г.

Москва 2026

Оглавление

1. Цель работы.....	3
2. Ход выполнения работы.....	3
2.1 Часть 1. Подготовка проекта к работе с БД.....	3
2.2 Часть 2. Превращение Document в JPA-сущность.....	3
2.3 Часть 3. Создание репозитория.....	5
2.4 Часть 4. Обновление сервиса.....	5
2.5 Часть 5. Создание View со списком документов.....	6
2.6 Часть 6. Тестирование приложения.....	7
2.7 Часть 7. Доработайте приложение.....	9
3. Вывод.....	12
4. Ответы на контрольные вопросы.....	13

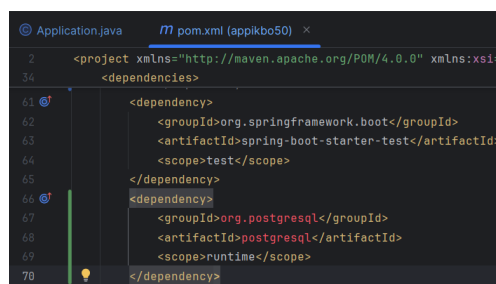
1. Цель работы

Превратить класс Document в полноценную сущность БД, создать репозиторий для управления данными и реализовать страницу со списком документов и функцией поиска.

2. Ход выполнения работы

2.1 Часть 1. Подготовка проекта к работе с БД

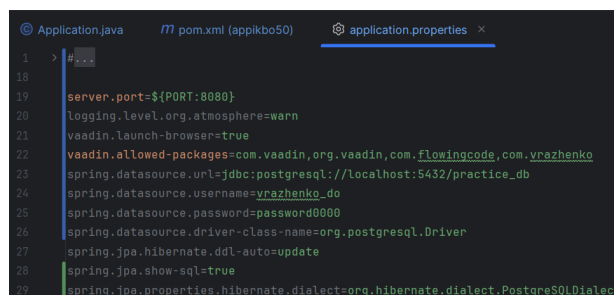
Результат добавления зависимостей для JPA и PostgreSQL представлен на Рисунке 1.



```
<?xml version="1.0" encoding="UTF-8" ?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.vrazhenko</groupId>
    <artifactId>document-feature</artifactId>
    <version>1.0.0</version>
    <packaging>war</packaging>
    <name>document-feature</name>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>
    </dependencies>
</project>
```

Рисунок 1 – Результат добавления зависимостей

Результат добавления параметров подключения к БД представлен на Рисунке 2.



```
#
server.port=${PORT:8080}
logging.level.org.atmosphere=warn
vaadin.launch-browser=true
vaadin.allowed-packages=com.vaadin.org.vaadin,com.floatingcode,com.vrazhenko
spring.datasource.url=jdbc:postgresql://localhost:5432/practice_db
spring.datasource.username=vrazhenko_do
spring.datasource.password=password0000
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

Рисунок 2 – Результат добавления параметров подключения к БД

2.2 Часть 2. Превращение Document в JPA-сущность

Содержание класса Document.java представлено в листинге ниже:

```
package com.vrazhenko.documentfeature;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
```

```

@Table(name = "documents")
public class Document {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "title", nullable = false)
    private String title;

    @Column(name = "reg_number", unique = true)
    private String regNumber;

    @Column(name = "reg_date")
    private LocalDate regDate;

    @Column(name = "type")
    private String type;

    @Column(name = "urgent")
    private boolean urgent;

    public Document() {
    }

    public Document(String title, String regNumber, LocalDate regDate,
String type, boolean urgent) {
        this.title = title;
        this.regNumber = regNumber;
        this.regDate = regDate;
        this.type = type;
        this.urgent = urgent;
    }

    // Геттеры и сеттеры
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getRegNumber() {
        return regNumber;
    }

    public void setRegNumber(String regNumber) {
        this.regNumber = regNumber;
    }

    public LocalDate getRegDate() {
        return regDate;
    }
}

```

```

    public void setRegDate(LocalDate regDate) {
        this.regDate = regDate;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }

    public boolean isUrgent() {
        return urgent;
    }

    public void setUrgent(boolean urgent) {
        this.urgent = urgent;
    }

    @Override
    public String toString() {
        return "Document{" +
            "id=" + id +
            ", title='" + title + '\'' +
            ", regNumber='" + regNumber + '\'' +
            ", regDate=" + regDate +
            ", type='" + type + '\'' +
            ", urgent=" + urgent +
            '}';
    }
}

```

2.3 Часть 3. Создание репозитория

Содержание интерфейса `DocumentRepository.java` представлено в ЛИСТИНГЕ НИЖЕ:

```

package com.vrazhenko.documentfeature;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface DocumentRepository extends JpaRepository<Document, Long> {

    // Поиск документов по части названия (без учета регистра)
    List<Document> findByTitleContainingIgnoreCase(String title);
}

```

2.4 Часть 4. Обновление сервиса

Содержание класса `DocumentService.java` представлено в ЛИСТИНГЕ НИЖЕ:

```

package com.vrazhenko.documentfeature;

import org.springframework.stereotype.Service;
import java.util.List;

@Service

```

```

public class DocumentService {

    private final DocumentRepository documentRepository;

    public DocumentService(DocumentRepository documentRepository) {
        this.documentRepository = documentRepository;
    }

    public Document saveDocument(Document document) {
        return documentRepository.save(document);
    }

    public List<Document> findAllDocuments() {
        return documentRepository.findAll();
    }

    public List<Document> findDocumentsByTitle(String title) {
        if (title == null || title.isBlank()) {
            return findAllDocuments();
        }
        return documentRepository.findByTitleContainingIgnoreCase(title);
    }
}

```

2.5 Часть 5. Создание View со списком документов

Содержание класса DocumentListView.java представлено в ЛИСТИНГЕ НИЖЕ:

```

package com.vrazhenko.documentfeature.ui;

import com.vaadin.flow.component.grid.Grid;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.data.value.ValueChangeMode;
import com.vaadin.flow.router.Route;
import com.vrazhenko.base.ui.MainLayout;
import com.vrazhenko.documentfeature.Document;
import com.vrazhenko.documentfeature.DocumentService;
import org.springframework.beans.factory.annotation.Autowired;

@Route(value = "documents", layout = MainLayout.class)
public class DocumentListView extends VerticalLayout {

    private final DocumentService documentService;

    private Grid<Document> grid = new Grid<>(Document.class);
    private TextField searchField = new TextField("Поиск по названию");

    public DocumentListView(@Autowired DocumentService documentService) {
        this.documentService = documentService;

        // Настройка поискового поля
        searchField.setPlaceholder("Введите часть названия...");
        searchField.setClearButtonVisible(true);
        searchField.setValueChangeMode(ValueChangeMode.LAZY);
        searchField.addValueChangeListener(e -> updateList());

        // Настройка колонок Grid
        configureGrid();
    }
}

```

```

        // Добавляем компоненты на форму
        add(searchField, grid);

        // Загружаем данные
        updateList();
    }

    private void configureGrid() {
        grid.removeAllColumns(); // убираем автоматические колонки
        grid.addColumn(Document::getTitle).setHeader("Название");
        grid.addColumn(Document::getRegNumber).setHeader("Регистрационный номер");
        grid.addColumn(Document::getRegDate).setHeader("Дата регистрации");
        grid.addColumn(Document::getType).setHeader("Тип документа");
        grid.setSizeFull();
    }

    private void updateList() {
        String searchTerm = searchField.getValue();
        grid.setItems(documentService.findDocumentsByTitle(searchTerm));
    }
}

```

2.6 Часть 6. Тестирование приложения

1. Проверка того, что PostgreSQL запущен (Рисунок 3):

```

PS C:\Additional\VirtualMachines\DatasAndDocs\vrazhenko_do> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAME
963138852c89   postgres:16   "docker-entrypoint.s..." 51 minutes ago Up 51 minutes 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
vrazhenko_do_postgres

```

Рисунок 3 – Проверка PostgreSQL

2. Запущенное приложение (Рисунок 4):

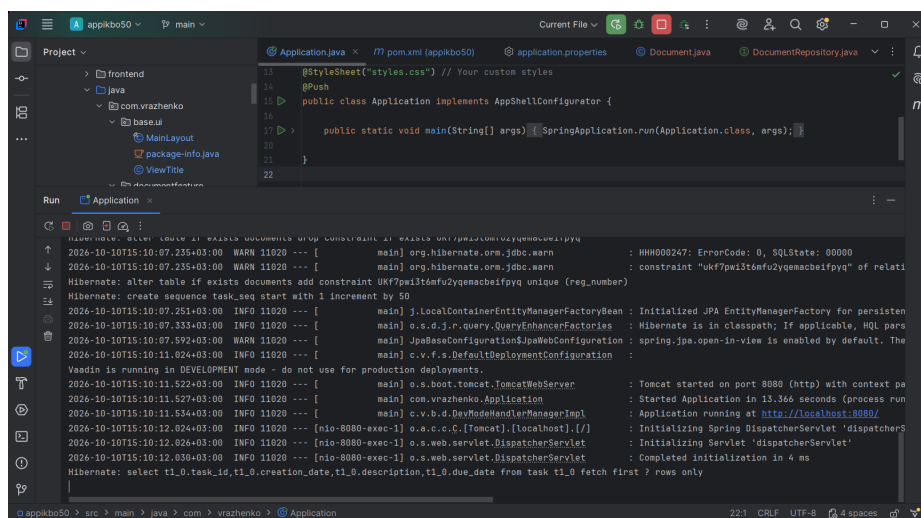


Рисунок 4 – Запуск приложения

3. Переход на <http://localhost:8080/documents> и пустая таблица (БД ещё пуста) (Рисунок 5):

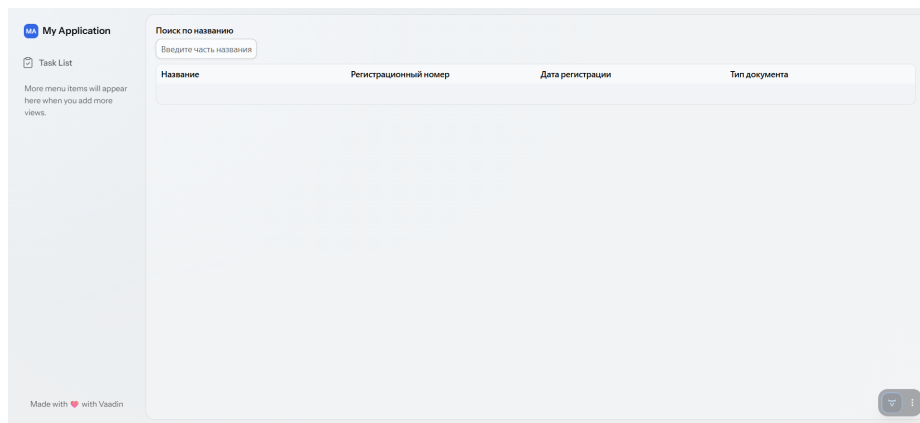


Рисунок 5 – Пустая таблица на сайте

4. Открытие формы <http://localhost:8080/document-form> и создание нескольких документов (Рисунок 6):

Рисунок 6 – Добавление документов

5. Наличие документов в таблице (Рисунок 7):

Название	Регистрационный номер	Дата регистрации	Тип документа
Приказ Враженко Д.О., ИКБО-50-23	100-01	2026-10-08	Приказ
Распоряжение Враженко Д.О., ИКБО-50-23	100-02	2026-10-09	Распоряжение
Служебная записка Враженко Д.О., ИКБО-50-23	100-03	2026-10-10	Служебная записка

Рисунок 7 – Запуск приложения

6. Отфильтрованная таблица (Рисунок 8):

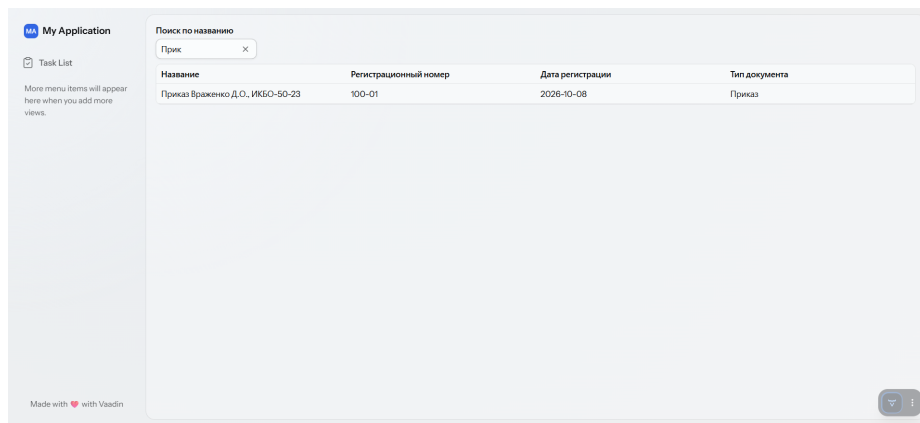


Рисунок 8 – Отфильтрованная таблица

2.7 Часть 7. Доработайте приложение

Обновлённое содержание интерфейса `DocumentRepository.java` представлено в листинге ниже:

```
package com.vrazhenko.documentfeature;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface DocumentRepository extends JpaRepository<Document, Long> {

    // Поиск документов по части названия (без учета регистра)
    List<Document> findByTitleContainingIgnoreCase(String title);

    // Поиск по типу документа
    List<Document> findByType(String type);

    // Комбинированный поиск по названию и типу
    List<Document> findByTitleContainingIgnoreCaseAndType(String title,
String type);
}
```

Обновлённое содержание класса `DocumentService.java`

представлено в листинге ниже:

```
package com.vrazhenko.documentfeature;

import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class DocumentService {

    private final DocumentRepository documentRepository;

    public DocumentService(DocumentRepository documentRepository) {
        this.documentRepository = documentRepository;
    }

    public Document saveDocument(Document document) {
        return documentRepository.save(document);
    }
}
```

```

    }

    public List<Document> findAllDocuments() {
        return documentRepository.findAll();
    }

    public List<Document> findDocumentsByTitle(String title) {
        if (title == null || title.isBlank()) {
            return findAllDocuments();
        }
        return documentRepository.findByTitleContainingIgnoreCase(title);
    }

    // Комбинированный поиск по названию + типу
    public List<Document> findDocuments(String title, String type) {
        boolean noTitle = (title == null || title.isBlank());
        boolean noType = (type == null || type.isBlank() ||
type.equals("Все"));

        if (noTitle && noType) {
            return documentRepository.findAll();
        } else if (noTitle) {
            return documentRepository.findByType(type);
        } else if (noType) {
            return
documentRepository.findByTitleContainingIgnoreCase(title);
        } else {
            return
documentRepository.findByTitleContainingIgnoreCaseAndType(title, type);
        }
    }
}

```

Обновлённое содержание класса DocumentListView.java

представлено в листинге ниже:

```

package com.vrazhenko.documentfeature.ui;

import com.vaadin.flow.component.combobox.ComboBox;
import com.vaadin.flow.component.grid.Grid;
import com.vaadin.flow.component.grid.GridSortOrder;
import com.vaadin.flow.component.orderedlayout.FlexComponent;
import com.vaadin.flow.component.orderedlayout.HorizontalLayout;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.data.provider.SortDirection;
import com.vaadin.flow.data.value.ValueChangeMode;
import com.vaadin.flow.router.Route;
import com.vrazhenko.base.ui.MainLayout;
import com.vrazhenko.documentfeature.Document;
import com.vrazhenko.documentfeature.DocumentService;
import org.springframework.beans.factory.annotation.Autowired;

import java.util.List;

@Route(value = "documents", layout = MainLayout.class)
public class DocumentListView extends VerticalLayout {

    private final DocumentService documentService;

    private final Grid<Document> grid = new Grid<>(Document.class);
    private final TextField searchField = new TextField("Поиск по

```

```

названию");
    private final ComboBox<String> typeFilter = new ComboBox<>("Тип
документа");

    public DocumentListView(@Autowired DocumentService documentService) {
        this.documentService = documentService;

        // Настройка поискового поля
        searchField.setPlaceholder("Введите часть названия...");
        searchField.setClearButtonVisible(true);
        searchField.setValueChangeMode(ValueChangeMode.LAZY);
        searchField.addValueChangeListener(e -> updateList());

        // Настройка фильтра по типу
        typeFilter.setItems("Все", "Приказ", "Распоряжение", "Служебная
записка");
        typeFilter.setValue("Все");
        typeFilter.setClearButtonVisible(false);
        typeFilter.addValueChangeListener(e -> updateList());

        // Панель фильтров в одну строку
        HorizontalLayout filters = new HorizontalLayout(searchField,
typeFilter);
        filters.setAlignItems(FlexComponent.Alignment.END);

        // Настройка колонок Grid
        configureGrid();

        // Добавляем компоненты на форму
        add(filters, grid);

        // Загружаем данные
        updateList();
    }

    private void configureGrid() {
        grid.removeAllColumns();

        // Колонка "Название" – с сортировкой
        grid.addColumn(Document::getTitle)
            .setHeader("Название")
            .setSortable(true)
            .setKey("title");

        grid.addColumn(Document::getRegNumber)
            .setHeader("Регистрационный номер");

        // Колонка "Дата регистрации" – с сортировкой
        grid.addColumn(Document::getRegDate)
            .setHeader("Дата регистрации")
            .setSortable(true)
            .setKey("regDate");

        grid.addColumn(Document::getType)
            .setHeader("Тип документа");

        // Новая колонка "Срочный документ"
        grid.addColumn(Document::isUrgent)
            .setHeader("Срочный документ")
            .setKey("urgent");

        grid.setSizeFull();
    }

```

```

        // Сортировка по умолчанию: новые документы сверху
        grid.sort(List.of(
            new GridSortOrder<>(grid.getColumnByKey("regDate"),
                SortDirection.DESENDING)
        ));
    }

    private void updateList() {
        String searchTerm = searchField.getValue();
        String selectedType = typeFilter.getValue();
        grid.setItems(documentService.findDocuments(searchTerm,
            selectedType));
    }
}

```

Результат изменений представлен на Рисунке 9:

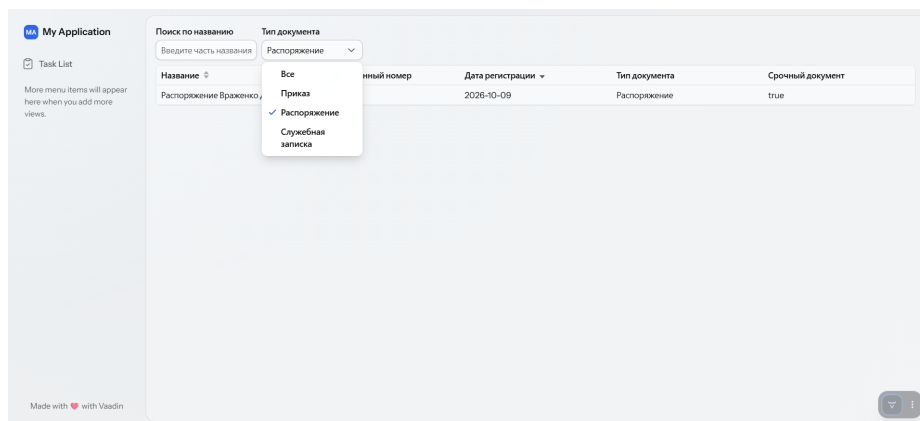


Рисунок 9 – Изменения в приложении

3. Вывод

В ходе практической работы класс Document был превращён в полноценную JPA-сущность с автогенерируемым первичным ключом, а хранение данных перенесено из оперативной памяти в реляционную базу PostgreSQL. Замена класса-заглушки DocumentRepository на интерфейс JpaRepository позволила отказаться от ручного написания CRUD-операций: Spring Data JPA автоматически сгенерировал реализацию методов save, findAll, findByTitleContainingIgnoreCase и других на основе имени метода. Слой сервиса (DocumentService) сохранил роль посредника между UI и репозиторием, инкапсулировав логику фильтрации и оставшись независимым от деталей хранения данных.

На стороне представления был реализован компонент Vaadin Grid, отображающий список документов с ленивой загрузкой, поиском по части названия, фильтром по типу документа и сортировкой по колонкам «Название» и «Дата регистрации». Данные успешно сохраняются в PostgreSQL через форму из ПР2 и сразу отображаются на странице /documents, что подтверждает корректную интеграцию всех слоёв приложения – от UI до базы данных.

Полученные навыки – работа с JPA-аннотациями, derived-запросами Spring Data, компонентом Grid и организацией слоистой архитектуры (Entity → Repository → Service → View) – являются базовыми при разработке корпоративных информационных систем: электронного документооборота, учётных систем, систем управления заявками и любых других приложений, где требуется надёжное хранение структурированных данных, гибкий поиск и удобное табличное представление.

4. Ответы на контрольные вопросы

1. Что такое JPA и какую роль в нем играет Hibernate?

JPA (Java Persistence API) – это спецификация (набор интерфейсов и аннотаций) для ORM в Java, описывающая, как объекты Java отображаются на таблицы БД. Hibernate – одна из реализаций этой спецификации, то есть библиотека, которая фактически выполняет преобразование объектов в SQL-запросы, управляет сессиями, кэшем и транзакциями. JPA задаёт «что делать», Hibernate – «как».

2. Каким требованиям должен удовлетворять POJO-класс, чтобы быть JPA-сущностью?

- Помечен аннотацией @Entity.
- Есть публичный или protected конструктор без аргументов.

- Класс не `final`, методы/поля не `final` (иначе проксирование Hibernate не работает).

- Поля `private` с геттерами/сеттерами.

- Обязательно наличие первичного ключа (`@Id`).

3. Что делает аннотация `@GeneratedValue(strategy = GenerationType.IDENTITY)`? Какие еще стратегии генерации ID вы знаете?

`IDENTITY` – значение ID генерируется самой БД при вставке строки (в PostgreSQL – через `SERIAL/GENERATED ... AS IDENTITY`). Другие стратегии:

- `AUTO` – выбор стратегии на усмотрение провайдера (обычно `SEQUENCE` или `TABLE`);

- `SEQUENCE` – значение берётся из последовательности БД (например, `CREATE SEQUENCE ...`);

- `TABLE` – отдельная таблица-счётчик, из которой Hibernate «выдаёт» ID.

4. В чем преимущество `JpaRepository` перед написанием JDBC-кода вручную?

- Готовые CRUD-методы (`save`, `findById`, `findAll`, `deleteById`, `count`) – не нужно писать SQL.

- Автоматическая генерация реализации Spring-ом во время запуска.

- Derived-запросы: имя метода → SQL-запрос автоматически.

- Управление транзакциями, кэшем, оптимистичными блокировками «из коробки».

- Меньше шаблонного кода, меньше ошибок, выше типобезопасность (всё на Java, а не на строках SQL).

5. Что такое «ленивая загрузка» (Lazy Loading) в компоненте Vaadin Grid и почему она критична для систем с миллионами документов?

Grid загружает из источника данных только те строки, которые в данный момент видны на экране, по мере прокрутки. Для миллиона документов это критично, потому что полная загрузка всех записей в память привела бы к переполнению JVM и огромным задержкам. Ленивая загрузка позволяет работать с любым объёмом данных при минимальном потреблении памяти.

6. Как реализовать сортировку по умолчанию в Grid (например, чтобы новые документы всегда были сверху)?

Через `GridSortOrder`:

```
grid.sort(List.of(
    new GridSortOrder<>(grid.getColumnByKey("regDate"),
        SortDirection.DESCENDING)
));
```

Колонке заранее задаётся `.setSortable(true)` и `.setKey("regDate")`. Vaadin применит сортировку к переданному в `setItems(...)` списку.

7. Зачем нужен слой Service, если можно вызвать Repository напрямую из UI-контроллера? Как это связано с принципами чистой архитектуры?

Service инкапсулирует бизнес-логику: правила фильтрации, валидацию, транзакции, комбинацию нескольких репозиториях. UI не должен знать о деталях хранения – это принцип разделения ответственности (SRP) и инверсии зависимостей из чистой архитектуры. Благодаря Service можно менять способ хранения данных (JPA, JDBC, REST), не затрагивая UI, а также легко писать unit-тесты с моками.

8. Как Spring Data JPA понимает, какой запрос выполнить для метода findByTitleContainingIgnoreCase?

По имени метода: `findBy` – `SELECT`, `Title` – поле сущности, `Containing` – условие `LIKE %...%`, `IgnoreCase` – `LOWER(...)` с обеих сторон. Spring парсит имя метода и на основе ключевых слов (`And`, `Or`, `Containing`, `IgnoreCase`, `Between` и т.п.) генерирует JPQL-запрос. Если поле названо иначе – будет ошибка `No property found`.

9. В чем разница между @Entity и @Table? Когда можно не указывать @Table?

`@Entity` помечает класс как JPA-сущность (обязательно). `@Table` задаёт имя таблицы, схему, индексы и ограничения – это необязательная настройка. `@Table` можно не указывать, если имя таблицы совпадает с именем класса (с учётом стратегии именования Hibernate, обычно – `camelCase` → `snake_case`). Например, класс `Document` без `@Table` отобразится на таблицу `document`.

10. Что произойдет, если в application.properties указать spring.jpa.hibernate.ddl-auto=validate, а таблица в БД отсутствует?

Приложение не запустится. Hibernate при старте проверит соответствие схемы БД сущностям и выбросит исключение `SchemaManagementException` со списком отсутствующих таблиц/колонок. Режим `validate` используется в продакшене, когда схемой управляют отдельно (например, через Flyway или Liquibase), чтобы избежать неожиданных изменений БД.